

AspireApp1 - Fynd och förslag

Bygge, lokal körning på macOS och analys av visualiseringsmöjligheter · Branch `claude/build-run-locally-wlecgw` · 30 september 2026

0 / 0

varningar / fel vid bygge

20 / 20

tester gröna

14

tjänster startade

7

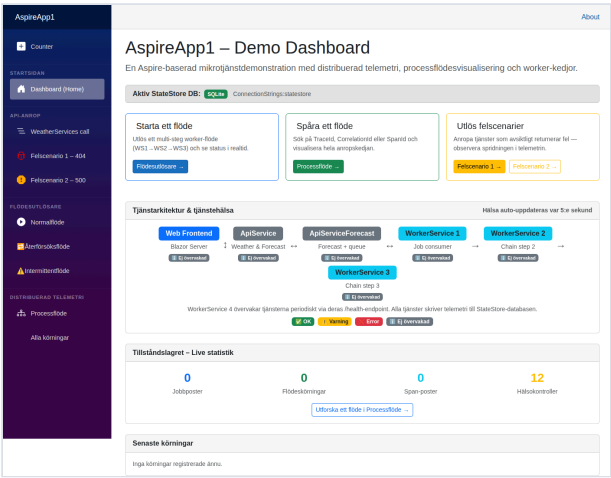
visualiseringsförslag

1. Sammanfattning

- Lösningen **bygger och körs** på .NET 10 SDK (10.0.401) med Aspire 13.5. Hela kedjan (AppHost, Web, API-tjänster och WorkerService 1-4) startar.
- SQLite är nu standard** för StateStore. SQL Server-standarden (`Trusted_Connection`) fungerade inte på macOS.
- Uppsättningen på iMac krävde tre åtgärder: nyare SDK, Aspire CLI och ett betrott HTTPS-certifikat.
- Appen visar i dag **en körning eller ett ögonblick i taget**. Stora delar av datan i StateStore används inte för trender eller mönster. Sju förslag beskrivs i avsnitt 5.

2. Bygge och test - fynd

Fynd	Orsak	Åtgärd
AppHost byggde inte (ASPIRE009)	AspireUseCliBundle=true kräver Aspire CLI.	Installera Aspire CLI (<code>dotnet tool install -g Aspire.Cli</code>).
dotnet test vägrar köra	Testprojektet använder Microsoft.Testing.Platform. VSTest-läget stöds inte på .NET 10 SDK.	Kör <code>dotnet run --project AspireApp1.Tests</code> .
1 test föll i container (UntrustedRoot)	Dev-certifikatet var inte betrott i miljön.	Miljöberoende. Alla 20 gick igenom vid senare körning.
Standard-DB fungerade inte på Mac	Server=.;Trusted_Connection=True är Windows-autentisering.	SQLite blev standard (se 4).



Web Frontend (Home) körd med SQLite. Skärmdump tagen med Playwright.

3. Lokal körning på iMac - fynd och lösningar

Symptom	Orsak	Lösning
A compatible .NET SDK was not found (10.0.303 krävs)	Endast 10.0.101 och 11 preview var installerade.	<code>dotnet-install.sh --channel 10.0 --install-dir ~/.dotnet</code>
<code>zsh: killed dotnet</code>	macOS kodsSignatur-cache efter att <code>~/.dotnet/dotnet</code> skrivits över på plats.	<code>cp</code> till ny fil, <code>rm</code> originalet, <code>mv</code> tillbaka.
<code>command not found: aspire</code>	Aspire CLI saknades, eller <code>~/.dotnet/tools</code> saknades i PATH.	Installera verktyget och lägg <code>~/.dotnet/tools</code> i <code>~/.zshrc</code> .
<code>error saving the HTTPS developer certificate</code>	Login-nyckelringen var låst.	<code>security unlock-keychain</code> och sedan <code>--clean</code> och <code>--trust</code> .
<code>authorization was denied since no user interaction was possible</code>	Session via SSH (Termius på iPad) . macOS kräver en ruta på skärmen, även med <code>sudo</code> , och även <code>authorizationdb</code> nekades (<code>-60005</code>).	Kör <code>dotnet dev-certs https --trust</code> i Terminal.app vid datorn. Löst.
<code>unknown file attribute: b</code>	<code>zsh</code> tolkar inte <code>#</code> -kommentarer vid inklistring.	Klistra in utan kommentarer, eller <code>setopt interactivecomments</code> .

Påverkan på andra projekt: SDK:er installeras sida vid sida. .NET 11 preview (Trading-appen) påverkas inte. Varje projekt väljer SDK via sin egen `global.json`.

Snabbstart (verifierad ordning)

```
bash ~/dotnet-install.sh --channel 10.0 --install-dir ~/.dotnet
dotnet tool install -g Aspire.Cli
echo 'export PATH="$PATH:$HOME/.dotnet/tools"' >> ~/.zshrc && source ~/.zshrc
dotnet dev-certs https --trust # i Terminal.app vid datorn, inte via SSH
git checkout claude/build-run-locally-wlecgw
aspire run
```

Via SSH utan betrott certifikat: `ASPIRE_ALLOW_UNSECURED_TRANSPORT=true dotnet run --project AspireApp1.AppHost --launch-profile http` och tunnla portarna 15259 (Dashboard) och 5004 (Web).

4. Genomförda ändringar (pushade)

Fil	Ändring
AppHost/appsettings.Development.json	StateStore:Provider = "Sqlite"
AppHost/AppHost.cs	Fallback-provider "Sqlite"
StateStore/StateStoreDbRegistration.cs	ResolveProvider: SQLite som standard, SQL Server endast om det anges uttryckligen. XML-kommentarer.
Tests/StateStoreDbRegistrationTests.cs	Testet heter nu ResolveProvider_DefaultsToSqlite.
.vscode/tasks.json, launch.json, extensions.json	⌘ B bygger, F5 startar AppHost, C# Dev Kit rekommenderas.
ReadMe.md, CLAUDE.md	Steg-för-steg-guide för lokal körning, felsökning av certifikat, SSH-körning, VS Code.

5. Visualisering - analys

Nuläge

Sida	Visar i dag
Home	Hårdkodat arkitekturdiagram, senaste hälsostatus per tjänst, fyra räknare, senaste körningar
FlowRuns	Lista med progress-bar och stegtidslinje per körning
ProcessFlow	Spanträd, Gantt-vy (utan tidsaxel eller förälder-barn-indrag), felmarkeringar, restart
Retry-/IntermittentDemo	Prickar för retry-försök per steg i en körning

Allt är byggt med Bootstrap. Det finns inget diagrambibliotek. **Inga trender över tid och inga mönster över många körningar.**

Outnyttjad data i StateStore

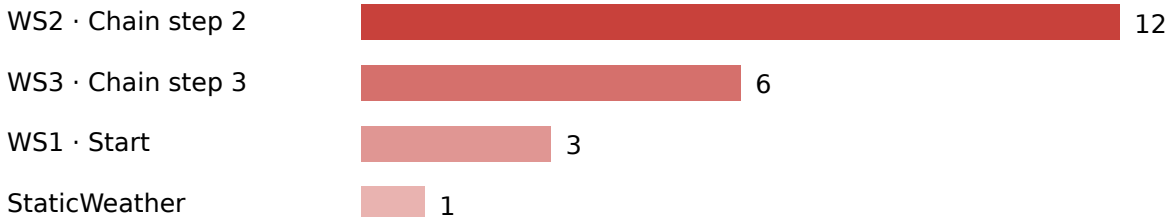
Tabell / fält	Används i dag	Potential
ServiceHealthRecords	Endast senaste raden	Upptid och historik: när och hur länge en tjänst var nere
Spans.ParentSpanId + ServiceName	Träd för en trace	Beroendediagram (vem anropar vem) härlett från riktig trafik
Spans.StartTime/EndTime	Gantt för en trace	Svarstider per tjänst (P50/P95), långsammaste operationerna
Spans.HttpStatusCode	Detaljvy	Felfördelning per tjänst och statuskod
FlowSteps.RetryAttempt	Prickar per körning	Retry-frekvens per steg över alla körningar
FlowRuns.Status/StartedAt	Lista	Körningar per tidsenhet, andel lyckade/misslyckade
JobStates	En räknare	Jobbkö-tavla: väntar, körs, klar, fel
ChainRuns	Visas inte alls	Egen vy för kedjekörningar

6. Förslag (prioriterade)

1. "Var fastnar flödena?" hög nytta liten insats

Stapeldiagram över vilket steg och vilken tjänst misslyckade körningar stannade på (senaste 24 h). Det besvarar appens huvudfråga direkt. Källa: `FlowSteps` med `Status = Failed`, grupperat på `StepName / ServiceName`.

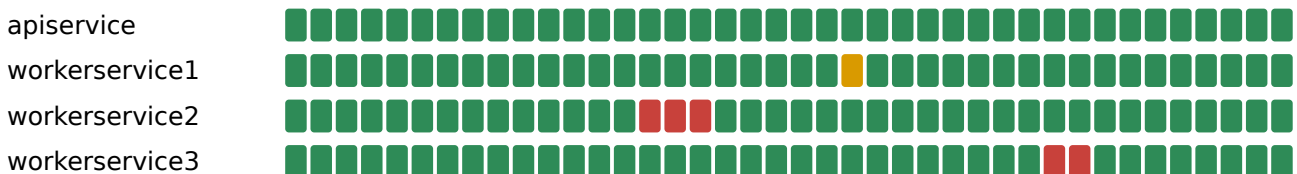
Skiss med exempeldata – antal misslyckade körningar per steg



2. Hälsotidslinje per tjänst hög nytta liten insats

En rad per tjänst med färgade rutor över tid. Man ser direkt när en tjänst var nere och om det sammanföll med fastnade flöden. Källa: `ServiceHealthRecords`, som `WorkerService4` redan skriver.

Skiss med exempeldata – 40 kontroller per tjänst



3. Levande beroendediagram hög nytta medelstor insats

Ersätt det hårdkodade arkitekturdiagrammet på Home med ett som räknas fram från spans (`ParentSpanId` → `ServiceName`). Varje pil visar antal anrop, felprocent och svarstid. Diagrammet stämmer alltid med verkligheten.

4. Trender för flödeskörningar medelstor nytta liten insats

Staplade staplar per 10 min eller timme: lyckade, misslyckade och pågående. Ger en bild av felfrekvensen över tid.

5. Svarstider per tjänst medelstor nytta medelstor insats

P50/P95 per tjänst och topp 10 långsammaste operationer. Pekar ut flaskhalsar i kedjan.

6. Bättre Gantt-vy i ProcessFlow medelstor nytta liten insats

- Tidsaxel med markeringar (0 s, 0,5 s, 1 s ...)
- Indrag efter förälder-barn, så att anropsträdet syns i tidslinjen
- Markering av den kritiska vägen

7. Retry-statistik, jobbkö och ChainRuns lägre nytta liten insats

Retry-försök per steg över alla körningar, statustavla för `JobStates` och en enkel vy för `ChainRuns`.

7. Teknikval

Rekommendation: inline-SVG i Razor-komponenter. Inga nya beroenden, passar Blazor Server, uppdateras med sidornas befintliga timer och fungerar offline. Förslag 1, 2, 4 och 6 är enkla SVG-staplar och SVG-rutor. För beroendediagrammet räcker en fast layout som beräknas i C#.

Alternativ: **Blazor-ApexCharts** (NuGet) om man vill ha zoom och tooltips. Det är ett extra beroende och behövs inte för att komma igång.

8. Rekommenderat nästa steg

1. Ny sida `/insights` med förslag 1 och 2. Mest nytta för minst kod.
2. Verifiera med Playwright-skärmdumpar och enhetstester för aggregeringsfrågorna (EF Core InMemory/SQLite).
3. Därefter förslag 3 (beroendediagram) och 6 (Gantt-förbättringar).